

Smart MCQ Solver

BSDA2001P: Introduction to DL and GenAI Project

submitted by:

Deepak Kumar Maurya (23f3004133)

GitHub: [deeepak00/smart-mcq-solver-dlgenai-2026](https://github.com/deepak00/smart-mcq-solver-dlgenai-2026)

1. Abstract / Executive Summary

This report covers my work on the Smart MCQ Solver Challenge on Kaggle — a five-option multiple-choice question answering task for a given prompts. For every question there will be a prompt and five candidate answers (A–E), and the goal is to rank the correct option as high as possible.

The competition uses **Mean Average Precision@3 (MAP@3)** as the scoring metric, so getting the right answer into the top three still earns partial credit even if it isn't ranked first.

I built and evaluated three quite different approaches end to end:

- TF-IDF + Logistic Regression on an expanded (prompt, option) representation,
- A custom deep-learning model that uses BPE tokenizer with a Conv1D + Bidirectional GRU + Attention-Pooling encoder
- A fine-tuned RoBERTa-Large transformer using Hugging Face's AutoModelForMultipleChoice head.

On out-of-fold cross-validation for all three models for [MAP@3](#)

- TF-IDF hit 0.9907 over 10 folds,
- the custom Conv-BiGRU model hit 0.9988 over 5 folds,
- RoBERTa-Large averaged 0.9038 over 5 folds.

2. Introduction

◆ 2.1 Problem Statement

The Smart MCQ Solver Challenge gives a dataset of multiple-choice questions, each with a text prompt and five answer options in columns A through E, plus a single correct-answer label. The task is to predict, per test question, an ordered top-3 list of the most likely correct options. Scoring is MAP@3, which rewards placing the correct answer first but still gives partial credit (1/2, 1/3) if it lands second or third.

◆ 2.2 Project Objective

Going in, I set myself three goals for this project:

- Build and fairly compare three genuinely different modelling approaches for MCQ answering — a classical sparse-feature baseline, a custom neural network trained completely from scratch, and a fine-tuned pretrained transformer.
- Practice and explore the full applied deep-learning workflow like EDA, tokenizer design, k-fold cross-validation, experiment tracking with Weights & Biases, GPU training on Kaggle, and ensembling for the final submission.
- Get comfortable with the Hugging Face ecosystem (AutoModelForMultipleChoice, the Trainer API, custom data collators) and see how it compares to writing a training loop by hand in plain PyTorch, which is what I did for the scratch model.

3. Dataset & Preprocessing

◆ 3.1 Dataset Description

The competition data is split into **train.csv** and **test.csv**. Every question has exactly 5 options. I checked and found zero missing values in the training set, so no imputation was needed.

Split	Number of Rows	Columns
train.csv	2,000	id, prompt, A, B, C, D, E, answer
test.csv	500	id, prompt, A, B, C, D, E

◆ 3.2 Exploratory Data Analysis (EDA)

Target balance:

- The correct answer is not equally distributed across A-E.
- B and C appear slightly more often, while D and E appear less often.
- The imbalance is mild, but it is worth considering when analyzing errors for each option.

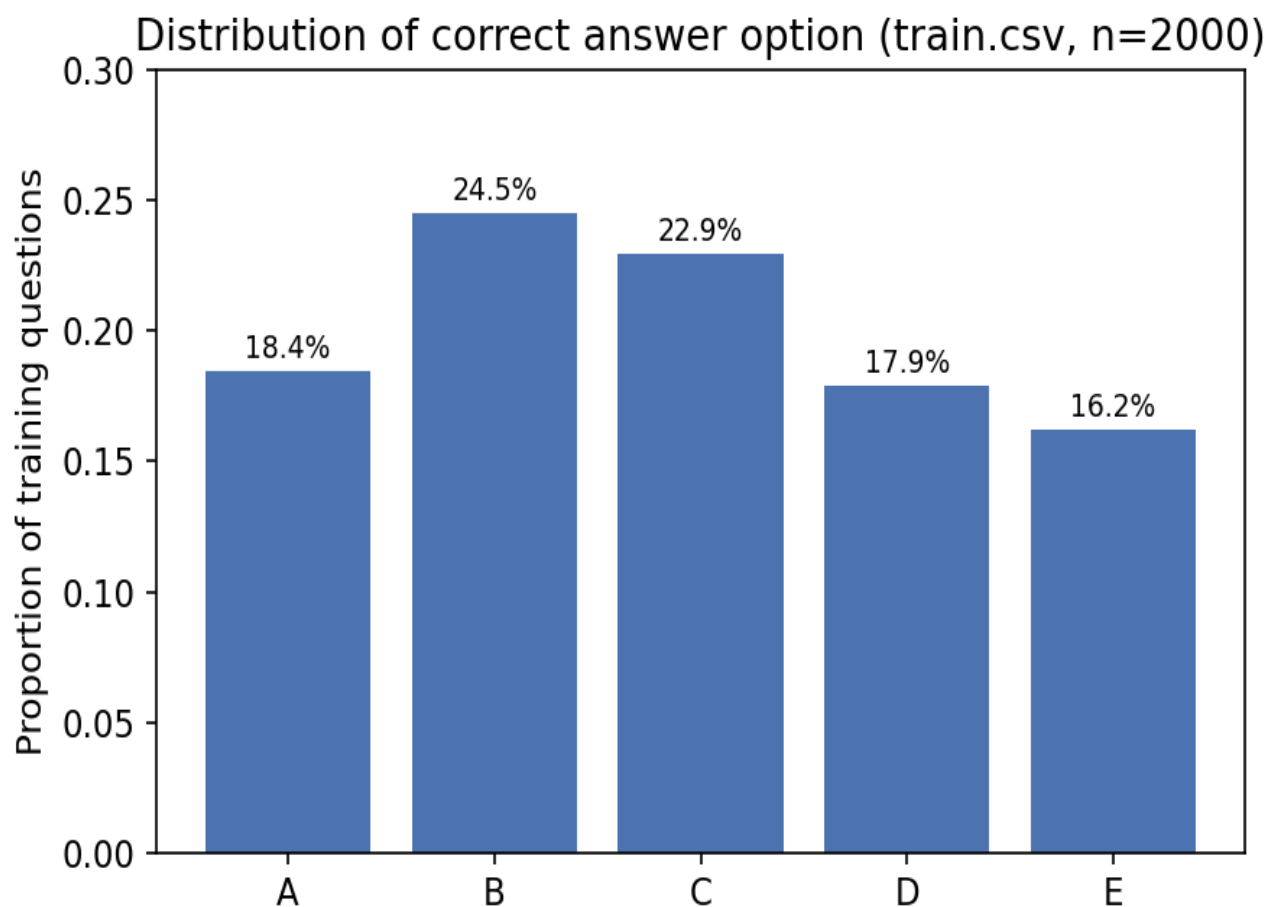


Figure 3.1 — Distribution of the correct answer option across the 2,000 training questions.

Text length:

- Prompts averaged **18 words**, while answer options averaged **26 words**.
- Since options were about **1.5× longer** and varied a lot in length, I used larger sequence limits to avoid cutting off important text.
- This led to **192 tokens for the scratch model** and **386 for RoBERTa**.

Statistic	Prompt length (words)	Avg. option length (words)
mean	18.15	26.28
std	6.78	17.03
min	3	1.0
50% (median)	17	23.0
max	51	85.4

Word-overlap check:

- The correct and incorrect options had almost identical word overlap with the prompt (4.11 vs. 4.15 words).
- This suggests that simple keyword matching is not a reliable shortcut for predicting the correct answer.
- Therefore, the task benefits from models that capture n-gram patterns or deeper semantic relationships.

◆ 3.3 Data Preprocessing

- No missing values were found, so no missing-value handling was needed.
- For the TF-IDF+ Logistic and scratch models, each question-option pair was treated as a separate binary prediction and later converted back into a top-3 ranking.
- RoBERTa used the multiple-choice format directly, keeping all 5 options together and predicting their scores jointly.
- Cross-validation was carefully split by questions to prevent the same question from appearing in both training and validation sets.

◆ 3.4 Data Augmentation

- I didn't use any synthetic data augmentation. The one augmentation-adjacent thing I did try was test-time augmentation (TTA) on the RoBERTa pipeline: I scored every test question twice — once with options in their original A-E order and once with the order reversed — then averaged the two probability vectors before taking the top-3. The idea was simply to reduce sensitivity to any positional bias the model might have accidentally picked up on option order.

4. Tokenization Strategy

◆ 4.1 Primary Tokenizer — RoBERTa Byte-Level BPE

- Byte-Level BPE, 50,265 tokens.
- Handles rare/unseen words effectively.
- Original tokenizer was retained because it is aligned with RoBERTa's pretrained embeddings.

◆ 4.2 Implementation Details

- Each question is represented as **5 prompt-option pairs**.
- Maximum sequence length: **386 tokens**, based on EDA.
- A custom `DataCollatorForMultipleChoice` keeps the five options together during training.

◆ 4.3 Experimental Comparison — Custom BPE

- A **BPE tokenizer was trained from scratch** for the scratch model.
- It learned about **5,700–5,800 tokens** from the small dataset.
- Maximum sequence length: **192 tokens**.
- Unlike RoBERTa, it had **no pretrained knowledge**, allowing a direct comparison between **pretrained vs. from-scratch learning**.

5. Modeling & Experimentation

I trained **three models**: TF-IDF baseline, custom **Conv-BiGRU**, and fine-tuned **RoBERTa**. The **BiGRU is part of the custom model**, not a separate RNN model. Therefore, I present the three models **once**, in the order they were developed and tested.

◆ 5.1 Model 1: TF-IDF + Logistic Regression

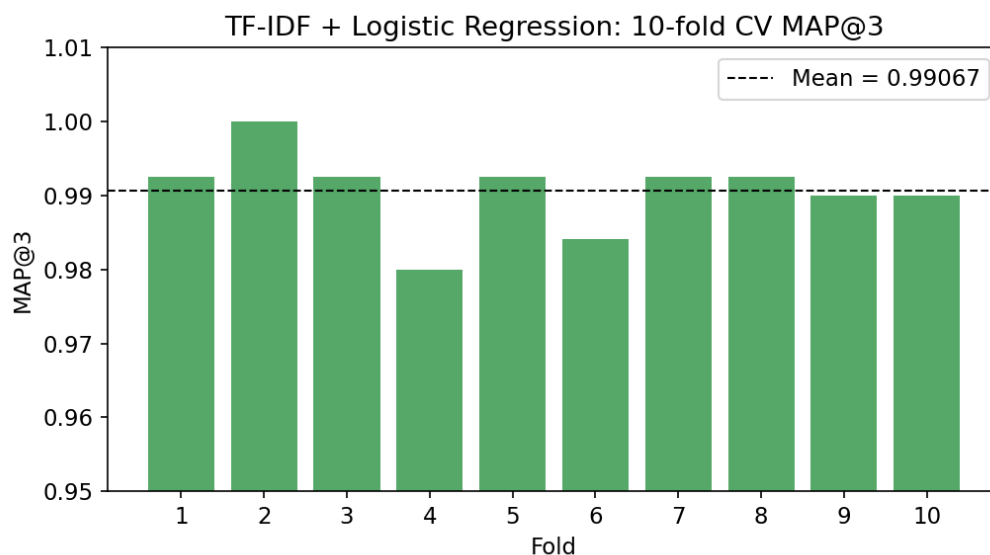
Architecture

- Each **prompt + option pair** is combined into one text and converted into **TF-IDF word n-grams (1–3)**.
- Used **sublinear TF**, `min_df=5`, `max_df=0.85`, and a **500k feature limit**. The final vocabulary contained **26,973 terms**.

- A **Logistic Regression** classifier ($C=10$, `class_weight='balanced'`) predicts whether each option is correct.
- The five option probabilities are ranked to produce the **top-3 answers**.

Key Points

- Using **1-3 word n-grams** helps capture meaningful phrases, not just individual words.
- It is a **simple, fast, and lightweight baseline** for comparison with the deep-learning models.
- The complete **10-fold cross-validation finished in under a minute on CPU**, making it very efficient to test.
- `saga` was used during cross-validation, while `lbfgs` was used for the final full-data training.

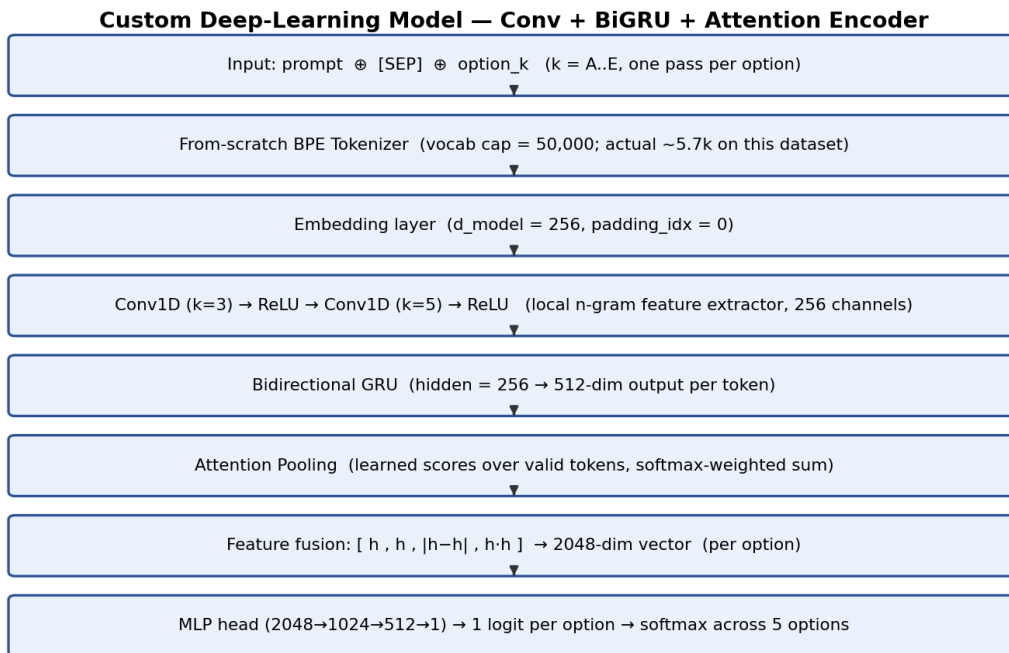


◆ 5.2 Model 2: Custom Deep-Learning Model — Conv1D + Bidirectional GRU + Attention

Architecture

- A custom **BPE tokenizer** feeds an embedding layer ($d_{model}=256$).
- Two **Conv1D layers** (kernel sizes 3 and 5) capture local word/phrase patterns.
- A **Bidirectional GRU** ($hidden=256$ per direction) captures sequence and word-order information.
- **Attention pooling** gives more weight to important tokens and produces one vector for each prompt-option pair.
- The resulting features pass through an **MLP head** ($1024 \rightarrow 512 \rightarrow 1$) to produce one score per option.

- The five option scores are trained together using **softmax cross-entropy with label smoothing (0.05)**.

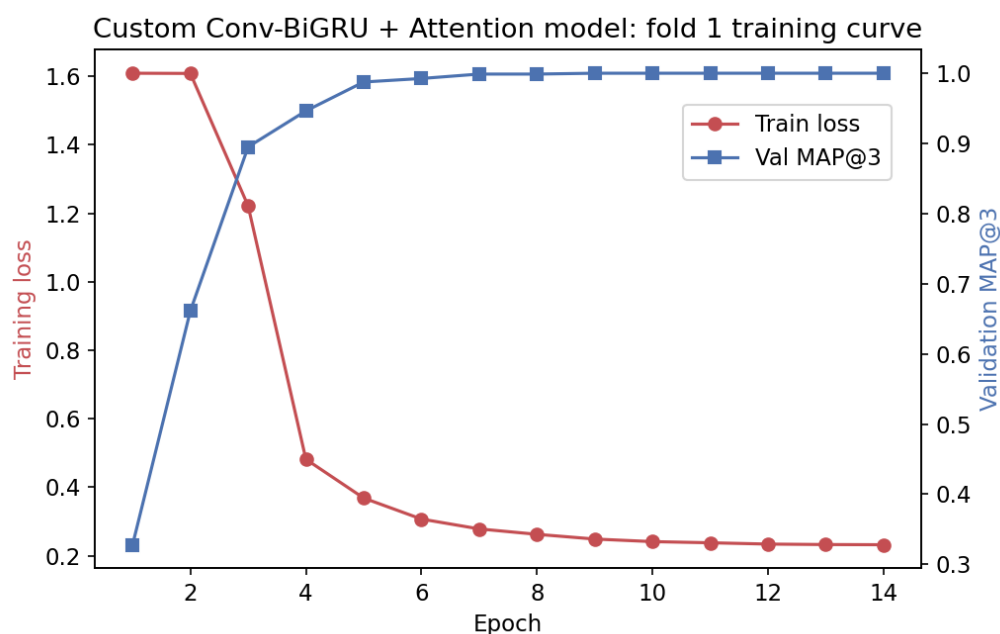


Key Points

- **Conv1D** captures local patterns, while **BiGRU** handles longer-range word order.
- **Attention pooling** focuses on important tokens instead of simply using the last or average hidden state.
- The model was trained with **AdamW (lr=2e-4, weight decay=0.02)**, cosine learning-rate scheduling, gradient clipping (1.0), mixed precision, and early stopping.
- Training used **5 stratified folds**, batch size **32**, and maximum sequence length **192**.
- The BiGRU is the **RNN component** of this model, so it is not a separate model.
- I used a **GRU instead of LSTM** because it is lighter and faster while still capturing sequence information.
- **Bidirectionality** allows each token to use information from both earlier and later words.

Important Limitation

- The current feature fusion [h , h , |h-h| , h·h] contains redundant terms because both sides come from the **same encoding**. Thus, |h-h| is always zero and h·h is simply h². This is a clear area for improvement.



Custom model, fold 1: training loss and validation MAP@3 across epochs. It reaches MAP@3 = 1.0 by around epoch 9, then early stopping kicks in a few epochs later.

◆ 5.3 Model 3: Fine-Tuned Transformer — RoBERTa-Large

Architecture

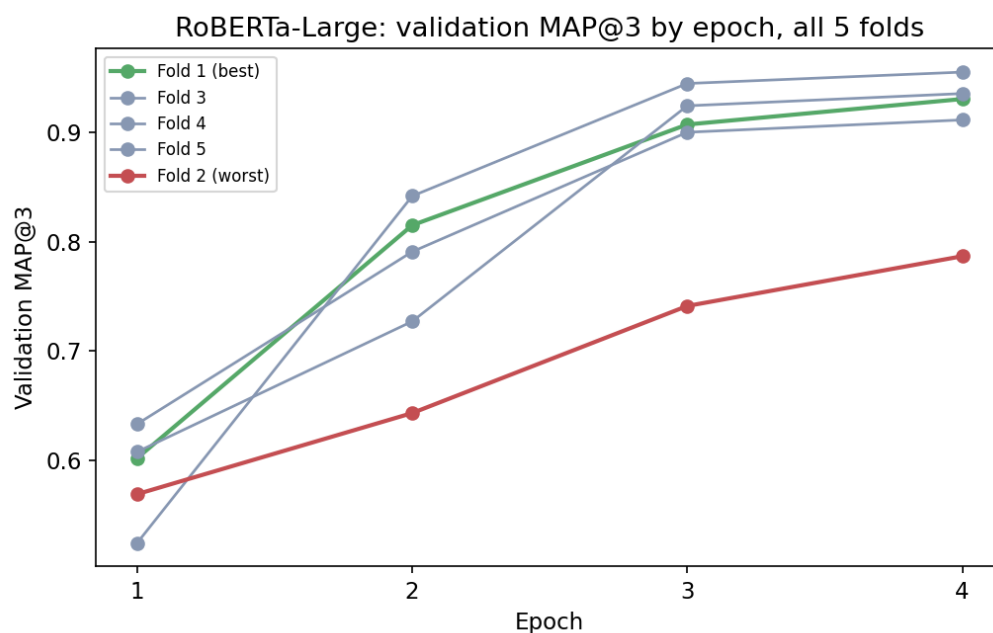
- RoBERTa-Large is an encoder-only Transformer with **24 layers**, **1024 hidden size**, **16 attention heads**, and **~355M parameters**.
- I used Hugging Face's **AutoModelForMultipleChoice**, where the same RoBERTa encoder scores all **5 prompt-option pairs**.
- The model produces **5 logits**, one for each option, which are compared using a **softmax and cross-entropy loss**.

Key Points

- **Self-attention** allows the model to connect words in the prompt and option, helping it capture **meaning and relationships**, not just word overlap.
- The same encoder is used for all 5 options, so their scores are **directly comparable**.
- This multiple-choice setup fits the task well because the model ranks the **5 options**, matching the **MAP@3** evaluation objective.
- RoBERTa's **pretrained knowledge** is especially useful for understanding rare words, technical terms, and semantic relationships.

Fine-Tuning Strategy

Hyperparameter	Value
Optimizer	AdamW
Learning rate	1e-5
LR schedule	Cosine, warmup_ratio = 0.15
Weight decay	0.01
Epochs	4 (early stopping, patience 2, on eval MAP@3)
Batch size	2 per device $\times$ 8 gradient-accumulation steps (effective 16)
Max sequence length	386 tokens
Precision	bf16 (auto-detected GPU support), fp16 fallback otherwise
Label smoothing	0.05
Cross-validation	StratifiedKFold, 5 folds, seed = 42
Frozen layers	None — all ~355M parameters fine-tuned



RoBERTa-Large validation MAP@3 by epoch, across all 5 folds. Every fold improves monotonically; fold 2 just converges to a noticeably lower plateau than the rest.

6. Performance & Comparative Analysis

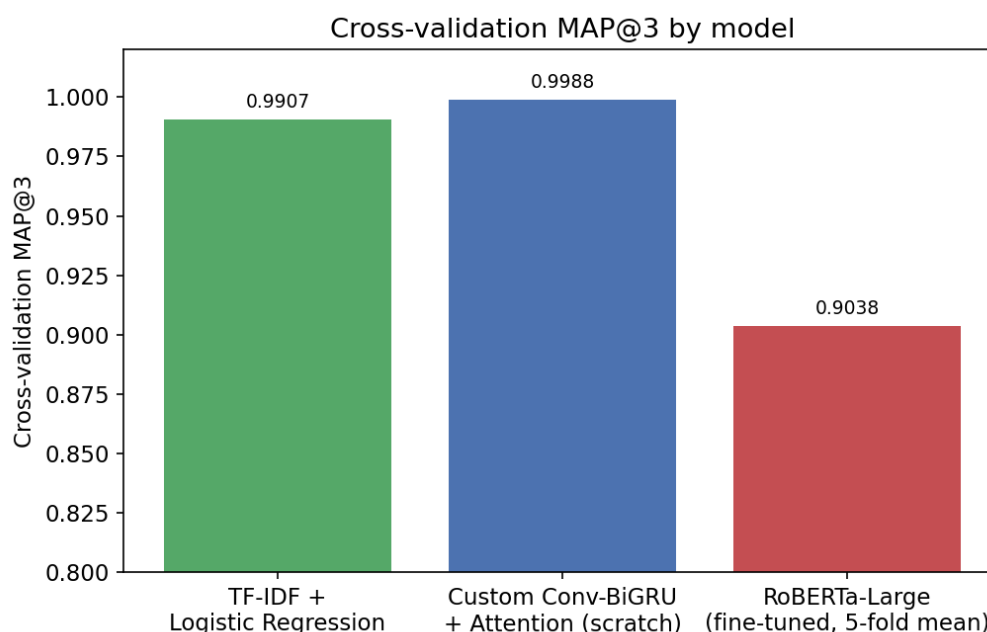
6.1 Evaluation Metrics

The competition's primary metric is MAP@3 (Mean Average Precision at 3): for each question the model submits an ordered top-3 list, and it scores 1.0 if the correct option is ranked first, 1/2 if second, 1/3 if third, and 0 otherwise — averaged over all questions.

6.2 Training Performance and Kaggle Performance

Model	CV scheme	CV MAP@3	Kaggle Leaderboard Score
TF-IDF + Logistic Regression	10-fold GroupKFold	0.9907	0.7544
Custom Conv + BiGRU + Attention	5-fold StratifiedKFold (OOF)	0.9988	0.7650
RoBERTa-Large (fine-tuned)	5-fold StratifiedKFold (mean)	0.9038	0.72

6.3 Comparative Report



- Classical and scratch models score higher, but this doesn't necessarily mean TF-IDF is better than RoBERTa.
- Their very high CV scores may be because the dataset has duplicate or very similar questions, making memorization easier.
- RoBERTa-Large is more expensive and gets lower, more varied scores, suggesting it struggles more with true generalization.
- Therefore, RoBERTa's lower score may actually be more realistic and trustworthy than the unusually high scores of the other models.

7. Conclusion & Future Work

7.1 Key Learnings

- A high CV score doesn't always mean a good model. Very high scores across different models made me question the data quality and possible leakage.
- On this small, domain-specific dataset, a simple scratch model performed as well as or better than RoBERTa-Large, showing that pretrained models don't always win on in-distribution data.
- Building both pipelines taught me the practical difference between using Hugging Face Trainer and writing a PyTorch training loop from scratch.

7.2 Challenges Faced

- **GPU limits:** RoBERTa-Large required gradient accumulation and checkpointing because a larger batch size did not fit on a single GPU.
- **Possible data leakage:** Extremely high CV scores led me to find near-duplicate questions, which may have inflated validation performance.
- **W&B reliability:** Occasional sync errors occurred during training, so remote logs had to be checked to ensure everything was uploaded correctly.

7.3 Areas for Improvement

- Remove near-duplicate questions before splitting to get a more reliable CV score.
- Improve the custom model architecture by separately encoding prompts and options for better interaction features.
- Ensemble different model families (TF-IDF, scratch model, and RoBERTa) to combine their strengths.
- Try DeBERTa-v3-base as a smaller, potentially more suitable pretrained model.
- Tune RoBERTa hyperparameters such as learning rate, warmup, and label smoothing to improve consistency.

8. References

- Hugging Face Transformers documentation — AutoModelForMultipleChoice / Trainer API: huggingface.co/docs/transformers
- scikit-learn documentation — TfidfVectorizer, LogisticRegression, StratifiedKFold/GroupKFold: scikit-learn.org
- PyTorch documentation — nn.GRU, nn.Conv1d, torch.amp: pytorch.org/docs
- Weights & Biases project (experiment tracking for all three pipelines): wandb.ai/23f3004133-dlgenai-2026/23f3004133-t22026
- Kaggle competition — Smart MCQ Solver Challenge: [/kaggle/competitions/smart-mcq-solver-challenge](https://kaggle.com/competitions/smart-mcq-solver-challenge)